

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 11 · LECTURE NOTES

Retrieval-Augmented Systems

Giving the model the information it does not have.

MODULE OUTCOMES

- Explain the architecture and what each stage contributes.
- Explain why retrieval quality dominates.
- Explain chunking and its trade-offs.
- Explain the failure modes specific to this design.
- Evaluate the retrieval and the generation separately.

1 The architecture

```
INDEXING (once, and on every update)
  split documents into chunks (section 3)
  embed each chunk, or index it for lexical search
  store with the metadata you will filter on

RETRIEVAL (per query)
  embed the query, or build a lexical query
  retrieve the top N candidates
  optionally rerank them with a stronger model
  (CSCE 670 Module 07)

GENERATION (per query)
  place the retrieved chunks in the prompt
  instruct the model to answer FROM them, and to say
  so when they do not suffice
  return the answer together with its citations

AND THE CITATIONS ARE NOT OPTIONAL. They are what
makes the answer checkable, which is the whole
advantage of this design over asking the model
directly (Module 12 section 3).
```

THIS IS THE ANSWER TO “THE MODEL DOES NOT KNOW MY DATA”

It solves currency, privacy, and attribution together: the information can postdate the training cutoff, it can stay inside your own infrastructure, and the answer can be traced back to a specific document — three different problems, one architecture.

And it is far cheaper than fine-tuning as a way of injecting knowledge, because updating the knowledge means updating an index rather than retraining anything — and the update is incremental, immediate, and reversible.

What it does not solve is reasoning across many documents — a question genuinely requiring the synthesis of twenty sources retrieves a handful and answers confidently from those, with no indication that it saw a fraction of the relevant material.

Nor does it solve aggregation. 'How many of our contracts expire this year' is a database query pretending to be a retrieval question — and routing it to retrieval produces a confident wrong number, which is the worst available outcome. **Recognising which questions are really queries is a design responsibility**, and the system should route them elsewhere.

2 Retrieval dominates

IF THE RIGHT PASSAGE IS NOT RETRIEVED, NO MODEL CAN ANSWER CORRECTLY

The generation step can only work with what it is given — so retrieval recall is a hard ceiling on the whole system's accuracy, and no amount of prompt improvement raises it. This is arithmetic rather than a tendency.

Which means measuring the retrieval stage separately is essential — and a system reported only end-to-end cannot tell you which half to fix, so the debugging proceeds by guesswork and usually lands on the prompt.

And the practical finding is that hybrid retrieval beats either method alone: lexical search handles exact terms, identifiers, product codes, and rare words; dense search handles paraphrase and synonymy (CSCE 670 Module 06) — and real queries contain both.

So the effort order is: retrieval first, reranking second, prompt third — which is the exact reverse of how these systems are usually debugged, since the prompt is the most visible and most easily changed component and therefore attracts the attention.

- **Hybrid lexical plus dense retrieval**, which is **the single largest improvement available** and is cheap to implement — run both and combine the rankings.
- **A reranker over the top fifty candidates**, which is **more accurate than retrieval and far too slow to apply to the whole corpus** — which is exactly why the two-stage structure exists (CSCE 670 Module 07).
- **Query rewriting** — expanding with synonyms, decomposing a compound question, or resolving pronouns against the conversation history — **which matters a great deal in multi-turn use**, where 'what about last year' is unretrievable as written.

- **Metadata filtering**, so that date, source, and **permission constraints are applied before relevance rather than after** — see the note.
- **And checking whether the answer is in the corpus at all**, which is **the diagnosis people skip** and which accounts for a surprising share of 'the model is bad at this' complaints. **Permission filtering before retrieval is a security requirement rather than a feature: retrieving a document the user is not permitted to read and then summarising it for them is a disclosure**, and the summarisation does not launder it (CSCE 701 Module 03).

3 Chunking

- **Small chunks retrieve precisely and lose context** — the retrieved passage may be entirely accurate and **unintelligible without its surroundings**, referring to 'this provision' or 'the above table' with no antecedent in view.
- **Large chunks preserve context and retrieve imprecisely**, because **the embedding averages over several topics and consequently matches none of them well** — which is a direct consequence of representing a long passage as one vector.
- **So: split on structure wherever it exists** — sections, paragraphs, list items, table rows, function definitions — **rather than on a fixed token count, which cuts mid-sentence, mid-table, and mid-code-block** and produces chunks that are hard to retrieve and useless when retrieved.
- **Overlap adjacent chunks**, so that **a fact spanning a boundary appears whole in at least one of them** — cheap insurance, at the cost of some index size.
- **And retrieve the chunk, then expand to its neighbours** before placing it in the prompt — **which gets precision in the retrieval and context in the generation. Retrieve small, generate large is the pattern worth adopting**, because **it resolves the trade rather than compromising on it**, and it is a small amount of code.

4 The failure modes

Failure	The cause	The fix
The passage was not retrieved	Retrieval recall.	Hybrid search and reranking (§2) — and check the answer is in the corpus.
It answers anyway	No abstention instruction, or no training that rewarded abstention.	Instruct it to decline, and <i>measure</i> abstention — see the note.
It ignores the retrieved context	Parametric knowledge overriding the provided passage.	Instruct it to prefer the source, and require citations so the override is visible.
Lost in the middle	Position effects in long contexts — material in the middle is used less.	Fewer, better chunks; and order them by rank rather than by document order.
Contradicting sources	The corpus genuinely disagrees with itself.	Surface the conflict rather than silently picking one — which is the honest behaviour.
Stale index	The documents changed and the index did not.	Treat reindexing as an operational process with monitoring, not a one-off script.

'Answers anyway' is the most dangerous row. A confident answer assembled from irrelevant retrieved passages is worse than a refusal, because it is both wrong and apparently sourced — the citations lend it credibility it has not earned. **And abstention has to be measured rather than assumed**, by deliberately retrieving irrelevant context and checking what happens, which is §4's third evaluation.

AND EVALUATE THE TWO STAGES SEPARATELY

Retrieval: recall at k, measured on a set of query-document pairs that you have labelled — which requires actually building that set, and **is the work people avoid**, with the result that they cannot tell retrieval failures from generation failures.

Generation, given correct context: is the answer right when the right passage was retrieved? — which isolates generation quality from retrieval quality and is straightforward once you have the labelled set.

Abstention: does it decline when the retrieved passages do not contain the answer? — tested by deliberately supplying irrelevant context and measuring how often it answers anyway. **This number is usually worse than people expect.**

And attribution: do the citations actually support the claims they are attached to? Which must be checked by reading, cannot be automated reliably, and **is where these systems most often fail quietly** — a plausible answer with a citation to a document that does not say that is the characteristic defect, and nothing in the pipeline catches it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lewis et al. — Retrieval-Augmented Generation (free)	§1 in the original — and worth reading for how much of the current practice was already there.
Liu et al. — Lost in the Middle (free)	§4's fourth row, measured — the position effect across long contexts, which has direct design consequences.
Gao et al. — Retrieval-Augmented Generation: a survey (free)	§2 and §3's design space , organised — useful for finding the variant that matches your constraints.
Rashkin et al. — Measuring Attribution (free)	§4's last evaluation — a framework for whether a citation actually supports a claim.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build the three-stage pipeline** over a document collection you have.
- 2 Require citations** and check ten answers against their sources.
- 3 Find a question your corpus cannot answer** and see what the system does.

- 4 **Label fifty query-document pairs** and measure retrieval recall at 5 and at 20.
- 5 **Add lexical search alongside dense** and remeasure.
- 6 **Add a reranker over the top fifty** and remeasure again.
- 7 **Compare fixed-size and structure-based chunking** on retrieval precision.
- 8 **Implement retrieve-small-generate-large** and compare.
- 9 **Supply deliberately irrelevant context** and measure how often it answers anyway.
- 10 **Check permission filtering happens before retrieval**, and fix it if not.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the three stages and what each decides.
- 2 Why are citations the point?
- 3 What three problems does this solve, and what two does it not?
- 4 Why is retrieval recall a hard ceiling?
- 5 Why does hybrid retrieval win, and what does each half contribute?
- 6 Give five ways to improve retrieval, and the security requirement.
- 7 State the chunking trade and the pattern that resolves it.
- 8 Name six failure modes with their causes and fixes.
- 9 Why is 'answers anyway' the most dangerous?
- 10 Give the four separate evaluations.